Transparency in DDBMS

Transparency is a feature of a **Distributed Database Management System (DDBMS)** that hides the internal implementation details of the distributed database from users. It allows users to work with the database as if it were a **single centralized database**, even though the data is actually stored at different locations.

Easy Definition

Transparency means hiding the complexity of a distributed database from the user.

Easy Example

Imagine a **university** has three campus servers:

- **Dhaka Server**
- **Chattogram Server**
- **Rajshahi Server**

Student information is distributed across these three servers.

A teacher writes:

```
SELECT * FROM Student;
```

The teacher does **not** need to know:

- Which server stores the data.
- How the data is divided.
- Whether backup copies exist.
- How the servers communicate.

The **DDBMS automatically collects the required data from all servers and displays one complete result.**

☞ **To the teacher, it looks like all data is stored in one single database.**

This is called Transparency.

Main Types of Transparency

There are **four main types** of transparency in DDBMS:

1. Distribution Transparency
2. Transaction Transparency
3. Performance Transparency
4. DBMS Transparency

1. Distribution Transparency

Distribution Transparency hides all the details of how the database is distributed. It hides **where the data is stored, how it is fragmented, and whether replica (copy) data exists.**

Easy Definition (1 Line)

Users can access data without knowing where or how the data is stored.

Easy Example

A university has three campus servers:

- Dhaka
- Chattogram
- Rajshahi

A teacher writes:

```
SELECT * FROM Student;
```

The teacher does **not** know:

- Which server stores the data.
- Whether the data is divided into fragments.
- Whether backup copies exist.

The **DDBMS automatically retrieves data from all necessary servers and shows one complete result.**

☞ **The teacher feels that all data comes from one database.**

Key Point

User gets the data without knowing where or how it is stored.

2. Transaction Transparency

Transaction Transparency ensures that every distributed transaction is completed correctly while maintaining database consistency.

Easy Definition (1 Line)

Either all operations are completed successfully, or none of them are completed.

Easy Example

Rahim transfers **৳5,000** from **Account A** to **Account B**.

This transaction has two steps:

1. Deduct ৳5,000 from Account A.
2. Add ৳5,000 to Account B.

If one server crashes before Step 2,

the DDBMS **cancels the entire transaction**.

Money is **never partially transferred**.

Key Point

Either everything happens, or nothing happens.

3. Performance Transparency

Performance Transparency ensures that the distributed database performs efficiently like a centralized database.

Easy Definition (1 Line)

The system remains fast even when many users access it at the same time.

Easy Example

During result publication,

100 students check their results simultaneously.

Instead of sending all requests to one server,

the DDBMS distributes the workload among different servers.

As a result,

everyone receives their results quickly.

4. DBMS Transparency

DBMS Transparency allows different types of DBMSs to work together under one common system.

Easy Example

Three university campuses use different DBMSs:

- Dhaka → MySQL
- Chattogram → Oracle
- Rajshahi → SQL Server

A teacher searches student information.

The teacher does not know which DBMS contains the data.

The DDBMS automatically retrieves the information from the correct DBMS.

Distribution Transparency

Distribution Transparency is a feature of a **Distributed Database Management System (DDBMS)** that hides the details of **data fragmentation, location, and replication** from users. Therefore, users can access the database as if it were a **single centralized database**.

Easy Definition (1 Line):
Distribution Transparency means users can use the database without knowing where or how the data is distributed.

Types of Distribution Transparency

There are **five types** of Distribution Transparency:

1. Fragmentation Transparency
2. Location Transparency
3. Replication Transparency
4. Local Mapping Transparency
5. Naming Transparency

1. Fragmentation Transparency

Fragmentation Transparency hides the fact that the database is divided into smaller parts (fragments). Users do not need to know how the data is fragmented or the fragment names.

Easy Example

Suppose the **Student** table is divided into two parts:

- Fragment 1 → Dhaka
- Fragment 2 → Chattogram

A teacher writes:

```
SELECT * FROM Student;
```

The teacher does not know that the table is divided into fragments. The **DDBMS automatically combines all fragments and shows the complete result.**

Easy Memory:
☞ **User does not know the data is divided.**

2. Location Transparency

Location Transparency hides the physical location of the data. Users may know the fragment name, but they do not need to know on which server it is stored.

Easy Example

The **Student** fragment is stored on the **Dhaka Server**.

The teacher requests the **Student** data.

The teacher does not know that the data is stored in Dhaka. The **DDBMS automatically finds the correct server and retrieves the data.**

Easy Memory:
☞ **User does not know where the data is stored.**

3. Replication Transparency

Replication Transparency hides the fact that multiple copies (replicas) of the same data are stored at different sites. Users do not know which copy is being used.

Easy Example

The **Student** table has copies on:

- Dhaka Server
- Rajshahi Server

The teacher writes:

```
SELECT * FROM Student;
```

The **DDBMS automatically selects one copy and returns the data.**

The teacher does not know that multiple copies exist.

4. Local Mapping Transparency

Local Mapping Transparency means the user knows both the **fragment name** and the **location** of the fragment. The user must specify both to access the data.

Easy Example

The teacher knows:

- Student Fragment → Dhaka Server

To retrieve data, the teacher specifies:

- Fragment Name: **Student**
- Location: **Dhaka**

The DDBMS then retrieves the data.

Easy Memory:
☞ **User knows both the fragment name and its location.**

5. Naming Transparency

Naming Transparency hides the actual database object names. Users access the database using **alias (simple) names**, while the DDBMS maps them to the real names.

Easy Example

Actual table name:

Student_DB_001

User writes:

```
SELECT * FROM Student;
```

Here, **Student** is an alias name.

The **DDBMS automatically finds the real table and returns the data.**

Easy Memory:
☞ **User uses an alias name instead of the real database object name.**

Transaction:

A transaction consists of a set of operations that perform a single logical unit of work in a database system and moves the database from one consistent state to another. It may be an entire program or a part of a program.

ACID Properties of Transactions

ACID is a set of **four important properties** that ensure every transaction is processed **correctly, safely, and reliably** in a database system.

ACID = Atomicity + Consistency + Isolation + Durability

Easy Definition (1 Line):
ACID properties ensure that every transaction is completed correctly without losing or corrupting data.

1. Atomicity (All or Nothing)

Definition

Atomicity means a transaction is treated as a **single unit of work**. Either **all operations are completed successfully, or none of them are performed**.

Easy Example

Rahim transfers **৳1,000** to Karim.

- Deduct ৳1,000 from Rahim's account.
- Add ৳1,000 to Karim's account.

If the second step fails, the first step is also cancelled.

☞ **Memory Trick: All or Nothing.**

বাংলা: সব হবে, না হলে কিছুই হবে না।

2. Consistency (Correct Data)

Definition

Consistency ensures that a transaction changes the database from **one valid (consistent) state to another valid state** by following all database rules.

Easy Example

Rahim has **৳5,000** in his account.

He cannot withdraw **৳10,000** because it breaks the database rule.

☞ **Memory Trick: Always Follow the Rules.**

বাংলা: Database সবসময় সঠিক থাকবে।

3. Isolation (Independent Execution)

Definition

Isolation means each transaction executes **independently**, and one transaction cannot see the incomplete work of another transaction.

Easy Example

Rahim is transferring money.

Before the transfer is completed, Karim cannot see the updated balance.

☞ **Memory Trick: Work Alone.**

বাংলা: একটি Transaction অন্যটির অসম্পূর্ণ কাজ দেখতে পারে না।

4. Durability (Permanent Result)

Definition

Durability means once a transaction is **committed**, its changes are **permanently stored** in the database, even if a system failure or power outage occurs.

Easy Example

You successfully transfer **৳2,000** using an ATM.

After receiving the **"Transaction Successful"** message, the power goes off.

The transaction is still saved in the database.

☞ **Memory Trick: Committed = Permanent.**

বাংলা: Commit হওয়ার পর Data স্থায়ীভাবে সংরক্ষিত থাকে।

Objectives of Distributed Transaction Management

The **Objectives of Distributed Transaction Management** are the goals of a DDBMS that ensure transactions are executed **efficiently, quickly, reliably, and with minimum communication cost**.

1. Improve CPU and Main Memory Utilization

The transaction manager should use the **CPU and main memory efficiently**. Since database applications spend much time waiting for **I/O operations**, the system should use special techniques to reduce idle time.

বাংলায়:
CPU এবং Main Memory যেন অযথা খালি না থাকে, সেগুলোকে সর্বোচ্চ কাজে ব্যবহার করতে হবে।

Easy Example

While one transaction is waiting for data from the disk, the CPU processes another transaction.

বাংলায়:
একটি Transaction Data-এর জন্য অপেক্ষা করছে, তখন CPU অন্য Transaction-এর কাজ করে।

☞ **Memory Trick: Don't keep the CPU idle.**
বাংলায়: CPU খালি রাখা যাবে না।

2. Minimize Response Time

The response time of each transaction should be **as short as possible** so that users receive results quickly.

বাংলায়:
প্রতিটি Transaction-এর Result যত দ্রুত সম্ভব User-এর কাছে পৌঁছাতে হবে।

Easy Example

A student checks the exam result and gets it in **2 seconds**.

বাংলায়:
একজন Student ২ সেকেন্ডে Result পেয়ে যায়।

☞ **Memory Trick: Fast Response.**
বাংলায়: দ্রুত ফলাফল।

3. Maximize Availability

The system should remain **available even if one site fails**. This helps in **transaction recovery** and **concurrency control**.

বাংলায়:
একটি Server নষ্ট হলেও System যেন কাজ চালিয়ে যেতে পারে।

Easy Example

If the **Dhaka Server** fails, users can still access data from the **Chattogram Server**.

বাংলায়:
ঢাকা Server বন্ধ হলেও চট্টগ্রাম Server থেকে Data পাওয়া যায়।

☞ **Memory Trick: Always Available.**
বাংলায়: সিস্টেম সবসময় চালু থাকবে।

4. Minimize Communication Cost

The transaction manager should reduce the number of **messages exchanged between different sites** to minimize communication cost.

বাংলায়:
বিভিন্ন Server-এর মধ্যে কম Message আদান-প্রদান করে খরচ কমাতে হবে।

Easy Example

Instead of sending **10 messages**, the system sends only **2 messages**.

বাংলায়:
১০টি Message-এর বদলে মাত্র ২টি Message পাঠানো হলো।

A Model for Transaction Management in DDBMS

A **Transaction Management Model** in a **Distributed Database Management System (DDBMS)** is a model that manages the execution of transactions while preserving the **ACID properties**. It ensures that transactions are executed **correctly, safely, and consistently** in a distributed database.

Easy Definition (1 Line):
It is a model that manages transactions in a distributed database while maintaining the ACID properties.

Types of Transactions

There are **two types of transactions** in DDBMS.

1. Local Transaction

A **Local Transaction** is a transaction that **accesses and updates data in only one local database (one site).**

Easy Example
A student updates his profile stored only in the **Dhaka Server**.

2. Global Transaction

A **Global Transaction** is a transaction that **accesses and updates data in more than one local database (multiple sites).**

Easy Example
A student transfers data from the **Dhaka Server** to the **Chattogram Server**.

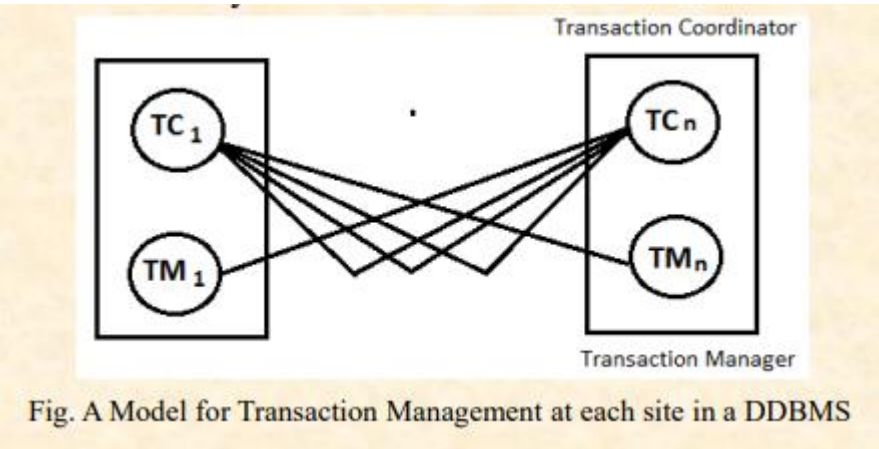


Fig. A Model for Transaction Management at each site in a DDBMS

Sub-Modules of Transaction Management Model

The Transaction Management Model consists of **two sub-modules**:

1. **Transaction Manager (TM)**
2. **Transaction Coordinator (TC)**

Transaction Manager (TM)

A **Transaction Manager (TM)** manages the execution of transactions at a **local site**. It may execute a **Local Transaction** or a part of a **Global Transaction**.

বাংলায়:
Transaction Manager (TM) একটি **Local Site-এর Transaction পরিচালনা করে।**

Responsibilities of Transaction Manager (TM)

1. Manage Local Transactions

It manages transactions that access data stored at the local site.

বাংলায়:
Local Site-এর Transaction পরিচালনা করে।

Example:
Dhaka Server-এর Student Data Update করা।

2. Maintain ACID Properties

It ensures that all transactions at the local site follow the **ACID properties**.

বাংলায়:
Local Transaction-এর ACID Properties বজায় রাখে।

3. Maintain Log for Recovery

It keeps a **log file** to recover data if a system failure occurs.

বাংলায়:
System Failure হলে Recovery করার জন্য Log সংরক্ষণ করে।

Example:
Power failure হলে Log দেখে Data পুনরুদ্ধার করা।

4. Participate in Concurrency Control

It helps coordinate the execution of multiple transactions running at the same time.

বাংলায়:
একই সময়ে একাধিক Transaction সঠিকভাবে চালাতে সাহায্য করে।

Example:
দুইজন Student একই সময়ে একই Data Update করতে চাইলে TM তা নিয়ন্ত্রণ করে।

Transaction Coordinator (TC) (Exam Ready Answer)

A **Transaction Coordinator (TC)** is a component of the **Transaction Management Model** in DDBMS. It **coordinates and controls the execution of both Local and Global Transactions** across different sites.

Responsibilities of Transaction Coordinator (TC)

1. Coordinate Local and Global Transactions

Definition

It coordinates the execution of both **Local Transactions** and **Global Transactions** initiated at a site.

বাংলায়:
একটি Site থেকে শুরু হওয়া Local ও Global Transaction-এর কাজ সমন্বয় করে।

2. Divide Transactions into Sub-transactions

It divides a **Global Transaction** into smaller **Sub-transactions** and sends them to the appropriate sites for execution.

3. Coordinate Commit or Abort

After all sub-transactions finish, TC decides whether the transaction will be **committed at all sites** or **aborted at all sites**.

বাংলায়:
সব Sub-transaction শেষ হলে TC সিদ্ধান্ত নেয় সব Server-এ **Commit** হবে, নাকি সব Server-এ **Abort** হবে।

Concurrency Control in DDBMS

Concurrency Control is the process of managing multiple transactions that access the database at the same time, ensuring that the database remains **correct, consistent, and accurate**.

বাংলায়:
Concurrency Control হলো একই সময়ে একাধিক Transaction-এর Data Access নিয়ন্ত্রণ করার প্রক্রিয়া, যাতে Database সঠিক থাকে।

Objectives of Distributed Concurrency Control

A good Distributed Concurrency Control mechanism has the following objectives:

1. Handle Site and Communication Failures

It should continue working even if a **site fails** or there is a **communication problem**.

বাংলায়:
Server বা Network সমস্যা হলেও System কাজ চালিয়ে যেতে হবে।

2. Allow Parallel Execution

It should allow **multiple transactions to execute simultaneously** for better performance.

বাংলায়:
একই সময়ে একাধিক Transaction চলতে দিতে হবে।

Easy Example

Two students check their results at the same time.

3. Reduce Overhead

Definition

It should use **minimum CPU, memory, and storage resources**.

বাংলায়:
কম CPU, Memory ও Storage ব্যবহার করতে হবে।

4. Work Well in a Network Environment

It should perform efficiently even when there is **network delay**.

বাংলায়:
Network ধীর হলেও System ভালোভাবে কাজ করবে।

☞ **Memory Trick: Good Performance in Network.**

5. Impose Few Constraints on Transactions

It should place **minimum restrictions** on transaction execution.

বাংলায়:
Transaction-এর উপর অপ্রয়োজনীয় বাধা দেওয়া যাবে না।

6. Preserve Data Consistency

It should always keep the **database consistent and correct**.

বাংলায়:
Database-এর Data সবসময় সঠিক রাখতে হবে।

Concurrency Control Anomalies

Concurrency Control Anomalies are the **problems that occur when multiple transactions access the same data at the same time** in a Distributed Database Management System (DDBMS).

বাংলায়:
একই সময়ে একাধিক Transaction একই Data ব্যবহার করলে যে সমস্যাগুলো হয়, সেগুলোকে Concurrency Control Anomalies বলে।

Types of Concurrency Control Anomalies

There are **five common anomalies**.

1. Lost Update Anomaly

A **Lost Update Anomaly** occurs when the update made by one transaction is **overwritten by another transaction**.

বাংলায়:
একটি Transaction-এর Update অন্য Transaction মুছে ফেলে।

Easy Example

- T1 updates Student Marks to **80**.
- T2 updates the same Marks to **90**.
- Final value becomes **90**, so **80 is lost**.

2. Uncommitted Dependency (Dirty Read)

This problem occurs when **one transaction reads data that has not yet been committed by another transaction**, and later that transaction is aborted.

বাংলায়:
একটি Transaction Commit হওয়ার আগেই অন্য Transaction সেই Data পড়ে ফেলে।

Easy Example

- T1 changes Balance to **5000** (not committed).
- T2 reads **5000**.
- T1 aborts.

Now T2 has read **wrong data**.

3. Inconsistent Analysis Anomaly

This anomaly occurs when **one transaction reads data while another transaction updates some of that data**, resulting in inconsistent results.

বাংলায়:
একটি Transaction Data পড়ছে, আর একই সময়ে অন্যটি সেই Data পরিবর্তন করছে।

Easy Example

A teacher calculates the **average marks**.

While calculating, another teacher updates one student's marks.

The calculated average becomes **incorrect**.

4. Phantom Read Anomaly

A **Phantom Read Anomaly** occurs when **a transaction reads some records, and before it finishes, another transaction inserts new records that satisfy the same condition**.

বাংলায়:
একটি Transaction Data পড়ার সময় অন্য Transaction নতুন Record যোগ করে।

Easy Example

T1 searches:

Students with marks **above 80**.

Before T1 finishes,

T2 inserts a new student with **85 marks**.

Now T1 gets an **extra record**.

5. Multiple-Copy Consistency Problem

This problem occurs when **the same data is stored at multiple sites**, but **only one copy is updated**, making the copies inconsistent.

বাংলায়:
একই Data-এর অনেক Copy থাকলেও সব Copy একসাথে Update না হলে সমস্যা হয়।

Easy Example

Student data is stored in:

- Dhaka Server
- Chattogram Server

Only the Dhaka copy is updated.

The Chattogram copy remains old.

Now the database becomes **inconsistent**.

Locking-based Concurrency Control Protocols

Locking-based Concurrency Control Protocol is a method used in **Distributed Database Management System (DDBMS)** to control the simultaneous access of data. Before accessing any data item, a transaction must obtain a **lock** on that data item. This lock prevents other transactions from modifying the same data and helps maintain **data consistency**.

Easy Definition (1 Line)

A Lock is a mechanism that controls access to data by allowing a transaction to lock the data before reading or writing it.

বাংলায়

Lock হলো এমন একটি ব্যবস্থা, যা কোনো Transaction-কে Data পড়া বা পরিবর্তন করার আগে অনুমতি (Lock) নিতে বাধ্য করে, যাতে Data ভুল না হয়।

Easy Example

Suppose **Rahim** wants to update a student's marks.

Before updating, Rahim gets a **lock** on that record.

Now **Karim** cannot update the same record until Rahim finishes.

বাংলায়:
রাহিম আগে Lock নেয়, তাই করিমকে অপেক্ষা করতে হয়।

Types of Locks

There are **two types of locks**:

1. **Read Lock (Shared Lock)**
2. **Write Lock (Exclusive Lock)**

1. Read Lock (Shared Lock)

Definition

A **Read Lock** allows a transaction to **read** the data but **cannot update** it. At the same time, **other transactions are also allowed to read** the same data because **read-read operations do not conflict**.

বাংলায়

Read Lock থাকলে শুধু Data পড়া যায়, পরিবর্তন করা যায় না। একই Data অন্যরাও পড়তে পারে।

Easy Example

Three students check the same exam result at the same time.

All of them can read the result because they are only reading.

2. Write Lock (Exclusive Lock)

A **Write Lock** allows a transaction to **read and update** the data. While a Write Lock is active, **no other transaction can read or update** that data because **read-write and write-write operations conflict**.

বাংলায়

Write Lock থাকলে Data পড়া ও পরিবর্তন করা যায়, কিন্তু অন্য কেউ সেই সময় Data পড়তে বা পরিবর্তন করতে পারে না।

Easy Example

Rahim updates a student's marks.

Until Rahim finishes, Karim cannot read or update the same record.

Basic Operation of Locking-based Concurrency Control Protocol (Exam Ready Answer) ⭐

Definition

In a **Distributed Database Management System (DDBMS)**, the **Lock Manager** is responsible for managing all locks. It checks every lock request and decides whether a transaction will **get the lock immediately or wait** until the lock becomes available.

Easy Definition (1 Line)

The Lock Manager controls all lock requests and decides whether a transaction can access the data or must wait.

বাংলায়

Lock Manager সব Lock নিয়ন্ত্রণ করে এবং সিদ্ধান্ত নেয় কোনো Transaction এখনই Data ব্যবহার করবে নাকি অপেক্ষা করবে।

Basic Operation (Working Steps)

Step 1: Lock Request

When a transaction wants to access a data item, it first **requests a lock**.

বাংলায়:
Transaction প্রথমে Data-এর জন্য Lock চায়।

Step 2: Request Sent to Lock Manager

The **Transaction Manager** sends the lock request to the **Lock Manager**.

বাংলায়:
Transaction Manager, Lock Manager-এর কাছে Lock Request পাঠায়।

Step 3: Lock Checking

The **Lock Manager** checks whether the requested data item is **already locked by another transaction**.

বাংলায়:
Lock Manager দেখে Data আগে থেকেই অন্য কোনো Transaction Lock করে রেখেছে কি না।

Step 4: If the Data is Locked

If the data is already locked and the requested lock is **not compatible**, the Lock Manager **does not grant the lock**. The transaction **waits** until the current lock is released.

বাংলায়:
যদি Data আগে থেকেই Lock করা থাকে, তাহলে নতুন Transaction অপেক্ষা করবে যতক্ষণ না আগের Lock ছাড়া হয়।

Step 5: If the Data is Free

If the data is **not locked**, the Lock Manager **grants the lock** and informs the Transaction Manager. Then the transaction starts its execution.

বাংলায়:
যদি Data-তে কোনো Lock না থাকে, তাহলে Lock Manager Lock দেয় এবং Transaction কাজ শুরু করে।

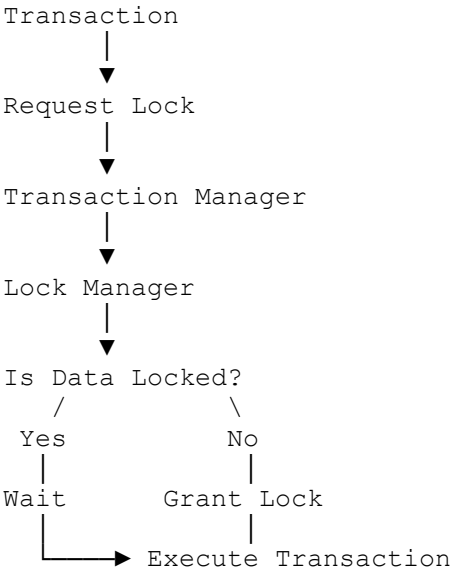
Easy Example

Suppose **T1** and **T2** both want to update the **Student Marks**.

- **T1** requests a **Write Lock**.
- The **Lock Manager grants the lock** to T1.
- Now **T2** also requests a Write Lock.
- Since T1 already holds the lock, **T2 must wait**.
- After T1 finishes and **releases the lock**, the Lock Manager gives the lock to **T2**.

বাংলায়:
T1 আগে Lock পায় এবং কাজ করে।
T2 পরে Lock চায়, তাই তাকে অপেক্ষা করতে হয়।
T1 কাজ শেষ করে Lock ছেড়ে দিলে T2 Lock পায়।

Flow Chart (Very Easy to Remember)



Up-gradation and Down-gradation of Lock (Exam Ready Answer)

In **Locking-based Concurrency Control Protocol**, a transaction may **change one type of lock into another type of lock**. This is called **Lock Conversion**. There are two types of lock conversion:

1. **Up-gradation of Lock**
2. **Down-gradation of Lock**

Easy Definition (1 Line):

Lock Conversion means changing one type of lock into another type during a transaction.

বাংলায়:

Lock Conversion অর্থ হলো একটি Lock-কে অন্য ধরনের Lock-এ পরিবর্তন করা।

1. Up-gradation of Lock

In **Up-gradation of Lock**, a transaction first acquires a **Read Lock**, and later it is **converted into a Write Lock**.

বাংলায়:

প্রথমে **Read Lock** নেওয়া হয়, পরে সেটিকে **Write Lock**-এ পরিবর্তন করা হয়।

Easy Example

Rahim first **reads** a student's marks.
Later, he decides to **update** the marks.
So, the **Read Lock** is changed into a **Write Lock**.

2. Down-gradation of Lock

In **Down-gradation of Lock**, a transaction first acquires a **Write Lock**, and later it is **converted into a Read Lock**.

বাংলায়:

প্রথমে **Write Lock** নেওয়া হয়, পরে সেটিকে **Read Lock**-এ পরিবর্তন করা হয়।

Easy Example

Rahim updates the student's marks.
After finishing the update, he only wants to **read** the marks.
So, the **Write Lock** is changed into a **Read Lock**.

Two-Phase Locking (2PL) Protocol (Exam Ready Answer)

Two-Phase Locking (2PL) Protocol is a locking protocol used in **DDBMS** to maintain **data consistency** and prevent conflicts between transactions.

According to the **2PL Protocol**, **a transaction cannot acquire a new lock after it has released any lock.**

Easy Definition (1 Line)

2PL is a protocol where a transaction first acquires locks and later releases locks, but never acquires a new lock after releasing one.

বাংলায়

2PL এমন একটি Protocol যেখানে Transaction আগে Lock নেয়, পরে Lock ছাড়ে; কিন্তু একবার Lock ছেড়ে দিলে আর নতুন Lock নিতে পারে না।

Two Phases of 2PL

Every transaction is divided into **two phases**:

1. **Growing Phase**
2. **Shrinking Phase**

1. Growing Phase

In the **Growing Phase**, a transaction can **acquire (take) new locks** and access data, but **cannot release any lock**.

বাংলায়

Growing Phase-এ Transaction যত খুশি **নতুন Lock নিতে পারে**, কিন্তু **একটিও Lock ছাড়তে পারে না**।

Easy Example

Rahim needs **Student Data** and **Result Data**.

- Takes Lock on Student Data ✓
- Takes Lock on Result Data ✓
- Cannot release any lock ✗

2. Shrinking Phase

In the **Shrinking Phase**, a transaction can **release locks**, but **cannot acquire any new lock**.

বাংলায়

Shrinking Phase-এ Transaction **শুধু Lock ছাড়তে পারে**, কিন্তু **নতুন কোনো Lock নিতে পারে না**।

Easy Example

Rahim finishes his work.

- Releases Student Lock ✓
- Releases Result Lock ✓
- Cannot take any new Lock ✗

Deadlock

A **Deadlock** is a situation in a database where **two or more transactions wait for each other forever**, so none of them can continue its execution.

Easy Definition (1 Line)

Deadlock occurs when transactions keep waiting for each other indefinitely.

বাংলায়

যখন দুই বা ততোধিক Transaction একে অপরের জন্য অপেক্ষা করতে থাকে এবং কেউই কাজ শেষ করতে পারে না, তখন তাকে Deadlock বলে।

Easy Example

Suppose,

- **T1** needs **Data A**, but it is locked by **T2**.
- **T2** needs **Data B**, but it is locked by **T1**.

Now,

- T1 waits for T2.
- T2 waits for T1.

☞ Both keep waiting forever.

This is called Deadlock.

Wait-for Graph (WFG)

Definition

A **Wait-for Graph (WFG)** is a **directed graph** that shows which transaction is waiting for another transaction.

- **Node = Transaction**
- **Edge (Arrow) = Waiting Relationship**

If **Transaction Ti** is waiting for a lock held by **Transaction Tj**, then an edge is drawn from **Ti → Tj**.

Deadlock Detection Rule

A deadlock can be detected easily using a Wait-for Graph.

- **If the graph contains a cycle → Deadlock exists.**
- **If there is no cycle → No Deadlock.**

Centralized DBMS vs Distributed DBMS

In Centralized DBMS

Deadlock detection is **easy** because there is only **one database site**.

Only **one Local Wait-for Graph (LWFG)** is needed to detect deadlocks.

বাংলায়

Centralized DBMS-এ একটি মাত্র Site থাকে, তাই **একটি LWFG** দিয়েই Deadlock খুঁজে পাওয়া যায়।

In Distributed DBMS

Deadlock detection is **more difficult** because transactions may be waiting across **multiple sites**.

Therefore, drawing only the **Local Wait-for Graph (LWFG)** is **not enough**.

A **Global Wait-for Graph (GWFG)** must also be created.

বাংলায়
Distributed DBMS-এ Transaction বিভিন্ন Site-এ থাকে, তাই Deadlock বিভিন্ন Site জুড়ে হতে পারে। এজন্য শুধু **LWFG** যথেষ্ট নয়, **GWFG**-ও তৈরি করতে হয়।

LWFG (Local Wait-for Graph)

An **LWFG** shows the waiting relationships of transactions **within one local site**.
It is used to detect **local deadlocks**.

বাংলায়
LWFG শুধুমাত্র **একটি Site-এর Transaction** দেখায় এবং **Local Deadlock** শনাক্ত করে।

Easy Example
Site-1:

```
T1 → T2
```

এটি শুধু Site-1-এর তথ্য দেখায়।

GWFG (Global Wait-for Graph)

A **GWFG** combines the LWFGs of **all sites** to detect deadlocks in the entire distributed system.
An **LWFG is a part of the GWFG**.

বাংলায়
GWFG সব Site-এর LWFG একত্র করে তৈরি করা হয় এবং পুরো Distributed System-এর Deadlock শনাক্ত করে।
LWFG হলো GWFG-এর একটি অংশ।

Easy Example

Suppose there are **3 sites**.

- Site-1 has LWFG
- Site-2 has LWFG
- Site-3 has LWFG

Individually, none of them has a cycle.
But after combining them into a **GWFG**, a cycle appears.
☞ Therefore, a **Global Deadlock** exists.

Three Techniques of Deadlock Resolution

1. Distributed Deadlock Prevention

This method **prevents a deadlock before it occurs**. If the system detects that a transaction may cause a deadlock, it **aborts or restarts** one transaction.

Easy Example

T1 wants a lock held by T2.

The system predicts a deadlock.

☞ T1 is stopped before the deadlock happens.

বাংলায়

Deadlock হওয়ার আগেই System একটি Transaction বন্ধ করে দেয়, যাতে Deadlock না হয়।

2. Distributed Deadlock Avoidance

Definition

This method **avoids deadlocks** by checking whether granting a lock will create an unsafe state. A lock is granted **only if the system remains safe**.

Easy Example

Before giving a lock to T1, the system checks if it is safe.

If it is safe → Lock is granted.

If not → T1 must wait.

বাংলায়

Lock দেওয়ার আগে System দেখে Deadlock হওয়ার সম্ভাবনা আছে কি না। নিরাপদ হলে Lock দেয়, না হলে অপেক্ষা করায়।

3. Distributed Deadlock Detection and Recovery

This method first **detects the deadlock** using a **Wait-for Graph (WFG)**. If a cycle is found, one transaction is selected as the **victim**, aborted, and restarted to recover from the deadlock.

Easy Example

T1 waits for T2.

T2 waits for T1.

A cycle is detected.

☞ T2 is aborted, and the deadlock is removed.

❑ **Distributed deadlock detection and recovery Method**

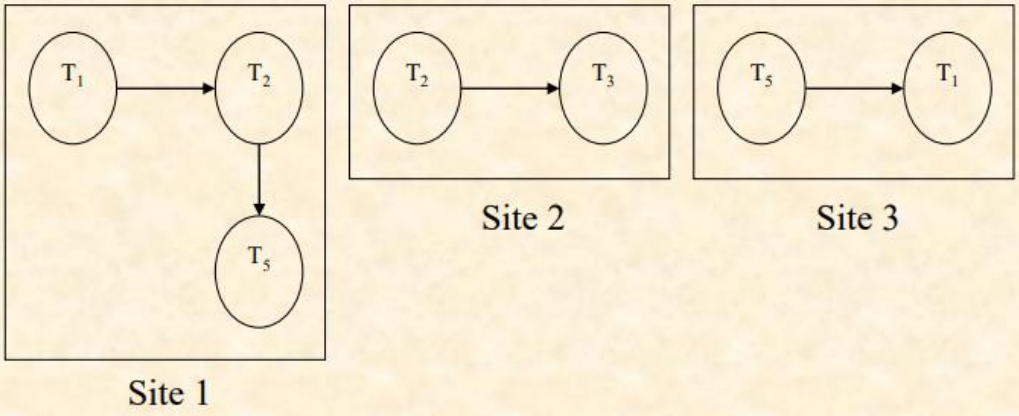


Fig. : Local Wait-For Graphs (LWFGs) at different Sites

❑ **Distributed deadlock detection and recovery Method**

The corresponding GWFG in Fig. illustrates that a deadlock has occurred in the distributed system, although no deadlock has occurred locally.

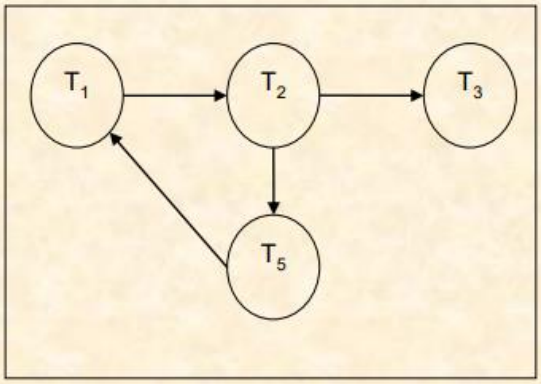


Fig.: Global Wait-For Graph (GWFG) for the above Fig.

বাংলায়

প্রথমে Wait-for Graph দিয়ে Deadlock খুঁজে বের করা হয়। তারপর একটি Transaction Abort করে Deadlock দূর করা হয়।

Types of Deadlock Prevention

1. Non-Preemptive Prevention

Definition

Non-Preemptive Prevention is a deadlock prevention technique in which the **requesting transaction (Ti)** is **aborted and restarted** if there is a risk of deadlock. The transaction holding the lock (**Tj**) is **not interrupted**.

Easy Example

Suppose **T1** requests a lock that is already held by **T2**.
The system detects a deadlock risk.

☞ **T1 is aborted and restarted**, while **T2 continues its work.**
This is called Non-Preemptive Prevention.

বাংলায় (বোঝার জন্য)

যদি **Lock চাওয়া Transaction (Ti)** Abort হয় এবং **Lock ধরে রাখা Transaction (Tj)** কাজ চালিয়ে যায়, তাহলে সেটি **Non-Preemptive Prevention**।

2. Preemptive Prevention

Preemptive Prevention is a deadlock prevention technique in which the **transaction holding the lock (Tj)** is **aborted and restarted** if there is a risk of deadlock. The lock is then given to the requesting transaction (**Ti**).

Easy Example

Suppose **T1** requests a lock that is already held by **T2**.
The system detects a deadlock risk.

☞ **T2 is aborted**, and the **lock is given to T1**.
This is called Preemptive Prevention.

বাংলায় (বোঝার জন্য)

যদি **Lock ধরে রাখা Transaction (Tj)** Abort হয় এবং **Lock চাওয়া Transaction (Ti)** Lock পেয়ে যায়, তাহলে সেটি **Preemptive Prevention**।

Centralized Deadlock Detection

Centralized Deadlock Detection is a method in which **one site** is selected as the **Deadlock Detection Coordinator (DDC)** for the entire distributed database system. The DDC collects all **Local Wait-for Graphs (LWFGs)** from different sites, combines them into a **Global Wait-for Graph (GWFG)**, and checks for cycles to detect deadlocks.

Easy Definition (1 Line)

Centralized Deadlock Detection uses one central site (DDC) to detect deadlocks by constructing a Global Wait-for Graph (GWFG).

বাংলায় (বোঝার জন্য)

একটি Central Site (**DDC**) সব Site-এর **LWFG** সংগ্রহ করে **GWFG** তৈরি করে এবং **Cycle** দেখে Deadlock শনাক্ত করে।

How Does It Work?

1. One site is selected as the **Deadlock Detection Coordinator (DDC)**.
2. Every site sends its **Local Wait-for Graph (LWFG)** to the DDC.
3. The DDC combines all LWFGs to create a **Global Wait-for Graph (GWFG)**.
4. The DDC checks the GWFG for **cycles**.
5. If a cycle is found, a **global deadlock** exists.
6. The DDC selects one transaction as the **victim**, aborts and restarts it to remove the deadlock.

Easy Example

Suppose there are **3 sites**.

- Site-1 sends LWFG.
- Site-2 sends LWFG.
- Site-3 sends LWFG.

The **DDC** combines them into one **GWFG**.
If the GWFG contains a **cycle**, the DDC aborts one transaction and removes the deadlock.

Advantages

- Simple to understand and implement.
- Easy to detect global deadlocks using one coordinator.

বাংলায়

- সহজ Method।
- একটি Coordinator দিয়েই পুরো System-এর Deadlock খুঁজে পাওয়া যায়।

Disadvantages

- If the **DDC fails**, deadlock detection stops.
- Communication cost is **high** because every site sends its LWFG to the DDC.
- It may detect **False Deadlocks**.
- It may also cause **Phantom Deadlocks**, leading to unnecessary rollback and restart.

Distributed Deadlock Detection

Distributed Deadlock Detection is a method in which **every site has its own deadlock detector**. Each detector creates a **Local Wait-for Graph (LWFG)** and cooperates with other sites to detect global deadlocks.

Easy Definition (1 Line)

Distributed Deadlock Detection uses a deadlock detector at every site to detect both local and global deadlocks.

বাংলায় (বোঝার জন্য)

প্রতিটি Site-এ একটি করে **Deadlock Detector** থাকে। সবাই মিলে Deadlock শনাক্ত করে।

How Does It Work?

1. Every site creates its own **LWFG**.
2. An **external node (Tex)** is added to each LWFG.
3. **Tex** represents transactions waiting between different sites.
4. The detector checks the LWFG for cycles.

- **Cycle without Tex → Local Deadlock.**
- **Cycle with Tex → Possible Global Deadlock.**

5. The LWFGs are exchanged and merged.
6. If the merged graph contains a cycle, a **Global Deadlock** is detected.

Easy Example

Suppose there are **3 sites**.

- Site-1 creates LWFG.
- Site-2 creates LWFG.
- Site-3 creates LWFG.

Each graph contains **Tex**.

After merging all LWFGs:

- If a **cycle** appears → **Global Deadlock**.
- If **no cycle** appears → **No Deadlock**.

1. False Deadlocks

A **False Deadlock** is a deadlock that is **incorrectly detected** because of **communication delay** between different sites in a distributed database system. In reality, no deadlock exists, but the system mistakenly thinks that a deadlock has occurred.

Easy Definition (1 Line)

False Deadlock is a deadlock detected due to delayed message transmission, even though no real deadlock exists.

বাংলায় (বোঝার জন্য)

Message দেরিতে পৌঁছানোর কারণে System ভুল করে Deadlock হয়েছে বলে মনে করে, যদিও আসলে Deadlock হয়নি।

How Does It Happen?

1. **Ti** is waiting for **Tj**.
2. Later, **Tj** releases the lock.
3. Due to **communication delay**, the detector does not receive this updated information in time.
4. The detector incorrectly thinks **Ti and Tj are still waiting for each other**.
5. As a result, it detects a **False Deadlock**.

Short Example

- **T1 waits for T2.**
- **T2 releases the lock.**
- The update message is delayed.
- The detector still thinks **T1 is waiting for T2**.

☞ **False Deadlock is detected.**

2. Phantom Deadlocks

A **Phantom Deadlock** occurs when the deadlock detector finds a **cycle in the Wait-for Graph** because it has **outdated information** about a transaction that has already been restarted. As a result, the detector reports a deadlock that no longer exists.

Easy Definition (1 Line)

Phantom Deadlock is a false deadlock caused by outdated information after a transaction has already been restarted.

বাংলায় (বোঝার জন্য)

একটি Transaction আগেই Restart হয়েছে, কিন্তু Detector সেই খবর না পেয়ে পুরনো তথ্য দিয়ে Deadlock শনাক্ত করে।

How Does It Happen?

1. **Ti** blocks another transaction.
2. **Ti** is restarted for another reason (not because of deadlock).
3. The restart message has not yet reached the deadlock detector.
4. The detector still sees **Ti** in the Wait-for Graph and finds a cycle.
5. It incorrectly detects a **Phantom Deadlock** and may restart another transaction unnecessarily.

Short Example

- **T1** is restarted.
- The detector has not received the restart message.
- It still sees **T1** in the Wait-for Graph.
- A cycle appears.

☞ **Phantom Deadlock is detected.**

False Deadlock	Phantom Deadlock
Caused by **message delay**	Caused by **outdated restart information**
No real deadlock exists	No real deadlock exists
Wrong detection due to delayed communication	Wrong detection because the detector has not received the restart message